FEDERAI — Sovereign AI for Resilient Public Decisions

- **Track:** Track 2B — Own Project
- **Event:** Hack Apertus Online Hack
- **Team:** FEDERAI — Hesam Salehpour
- **Demo:** See the video URL submitted with the project.

1. Summary

Public-sector decisions depend on policy documents containing objectives, responsibilities, evidence, assumptions, dependencies, and implementation risks. These elements are often difficult to review consistently across long documents. FEDERAI is an evidence-centred policy assurance prototype that uses the Apertus model family to extract structured findings from a submitted policy text and present them for human review. It checks whether quoted evidence occurs verbatim in the supplied source and organizes findings, relationships, limitations, and hypothetical stress scenarios into a review workflow. The prototype is designed to support decision-makers, not to replace legal, policy, or domain experts. It does not claim that a matching quotation proves a claim, that generated scenarios predict real outcomes, or that the current hosted inference configuration satisfies Swiss data-sovereignty requirements.

2. Architecture

FEDERAI consists of three Docker Compose services:

1. **Web application:** A React and TypeScript frontend built with TanStack Start. It provides the policy-analysis workflow and displays the structured results.
2. **API:** A Python FastAPI service that validates requests, invokes the configured Apertus inference endpoint, processes the response, and checks evidence quotations against the submitted text.
3. **Reverse proxy:** Nginx routes browser requests to the web application and API through a single exposed port.

The request flow is: policy text submission → API validation → Apertus inference → structured findings → exact-text evidence checks → results for human review.

The root `Makefile` delegates to `track_2b/Makefile`. The intended judge workflow is `make run` from the repository root, which builds and starts the Docker Compose services.

Target architecture and sovereignty

The application is containerized and could be adapted for deployment on an organization's own infrastructure, subject to installation, configuration, security review, and operational testing. However, the current configuration calls a hosted Public AI API endpoint. Its hosting location, jurisdiction, and data-residency guarantees have not been independently verified. Therefore, this submission does **not** claim that the tested configuration is already deployed on-premise, air-gapped, or in a verified sovereign Swiss cloud.

The inference endpoint is an external runtime dependency. Docker images and application

dependencies are built as part of the container workflow; model inference requires network access to the configured endpoint and valid credentials. The API token is supplied locally through an environment file and must not be committed to the repository.

3. Use of Apertus

- **Model:** `swiss-ai/apertus-v1.5-8b`
- **Use:** Inference for structured policy-document analysis.
- **Serving:** Hosted endpoint configured through environment variables.
- **Fine-tuning:** None.
- **Multi-agent orchestration:** None.

The backend sends the policy-analysis request to the configured chat-completions endpoint and processes the returned structured content. The application also applies a deterministic check to each extracted evidence quotation: whether the quoted string occurs in the submitted source text. This check is separate from model inference and does not require another model.

A successful exact-text match establishes only that the quotation is present in the submitted text. It does not establish that the quotation supports the associated claim, that the claim is true, or that the source itself is authoritative. Non-matching quotations are flagged for human review.

4. Data

The prototype is designed to analyse text supplied by the user. No external evaluation dataset is claimed for this submission. The repository's data directory contains project documentation rather than a large training corpus. A demonstration policy case should be treated as illustrative unless independently sourced and documented; it must not be presented as an official Swiss government assessment.

The model is not fine-tuned on submitted policy documents. Users should not submit confidential, personal, classified, or otherwise sensitive information to the current hosted configuration. Data-handling arrangements depend on the external inference provider and have not been independently audited as part of this prototype.

5. Evaluation

Evaluation to date is limited to implementation and smoke tests. No quantitative accuracy benchmark, comparative baseline, or statistically meaningful evaluation of policy-analysis quality is claimed.

Test	Method	Result
Docker Compose configuration	`docker compose config --quiet`	Passed locally
Web entry point	HTTP request to the integrated local stack	HTTP 200 observed
API health	`GET /api/health`	Reported application healthy and model configured

Frontend production build	Frontend build process	Completed successfully
Evidence verification	Deterministic exact-string comparison	Implemented; not a measure of factual correctness
Analysis quality	Quantitative expert-labelled benchmark	Not performed
Swiss data sovereignty	Independent verification of hosting and jurisdiction	Not verified

These results demonstrate basic integration, not the accuracy or operational suitability of FEDERAI for high-stakes public decisions. Model outputs may omit relevant information, misinterpret a source, or produce unsupported findings. Meaningful evaluation requires a labelled set of policy documents, expert review, metrics for extraction and evidence quality, and tests of failure cases.

6. Limitations

- Model-generated findings can be incomplete, incorrect, or sensitive to prompt and input variations.
- Exact-text matching verifies quotation presence only; it does not validate reasoning, entailment, truth, or source authority.
- Hypothetical scenarios are prompts for structured review, not forecasts or quantified impact estimates.
- No quantitative benchmark or expert-validated accuracy score is available.
- The hosted endpoint requires external connectivity and credentials.
- Swiss hosting, jurisdiction, and data residency are not verified.
- Production readiness, security certification, and suitability for confidential government documents have not been established.

Human reviewers must verify source context, assess the validity of each claim, and make all consequential decisions independently.

7. Reproducibility

From a clean checkout with Docker Compose and `make` installed:

1. Copy `track_2b/.env.example` to `track_2b/.env`.
2. Add a valid API token locally and confirm the configured endpoint and model name.
3. Run `make run` from the repository root.
4. Open `http://localhost:8000`.
5. Check `http://localhost:8000/api/health` for API status.
6. Stop the services with `docker compose down` from `track_2b/`.

A working API credential and external network access are required for live inference. The credential must remain local and must never be included in source control or demonstration recordings. Results may vary between model calls. The repository commit identifier should be recorded after the final submission commit.

8. Next steps

The next development phase should add an expert-labelled evaluation suite, test citation support beyond exact-string matching, measure robustness across policy-document formats, and improve uncertainty reporting. Deployment should be validated on organization-controlled infrastructure or a verified Swiss sovereign environment before processing sensitive documents. Additional work should include threat modelling, access controls, audit logging, data-retention controls, and usability testing with public-sector reviewers.

License

See the repository's `LICENSE` file and the Hack Apertus terms and conditions.

References

1. Hack Apertus — https://hackapertus.ch/
2. Apertus model family — https://huggingface.co/swiss-ai
3. FastAPI documentation — https://fastapi.tiangolo.com/
4. Docker Compose documentation — https://docs.docker.com/compose/
5. TanStack Start documentation — https://tanstack.com/start